TAKE-HOME PROJECT · AI-POWERED ALCOHOL LABEL VERIFICATION

Approach, Tools Used, and Assumptions

Austin Belman

Companion to the README, which covers setup, run, and deployment · Live demo: austin-belman-ttb-label-verifier-app.vercel.app

SUMMARY

The app checks an alcohol label image against the values the applicant submitted and the TTB rules. The design is two stages: a vision model **reads** the label, and deterministic Python **judges** it. Every field returns `pass` / `needs review` / `fail` with a reason, and the label image sits next to the verdicts so a reviewer can confirm any read in one glance.

FRONTEND	Next.js (App Router) + TypeScript · Tailwind CSS v3 · SheetJS for client-side Excel
API	FastAPI + Pydantic — one thin serverless function
ENGINE	Plain Python: `extraction.py`, `verification.py`, `config.py` · RapidFuzz
AI	OpenAI vision — `gpt-5.4-mini` (extraction), `gpt-4.1` (warning supplement)
DEPLOY	Vercel — a single project serves the frontend and the Python API
TESTS / CI	pytest · node:test · GitHub Actions on every PR and push to main

WHAT THE CUSTOMERS SAID THEY WANTED

- **Sarah (Deputy Director):** results in ~5 seconds — the last vendor pilot sometimes took 30–40 seconds per label and agents went back to eyeballing. A UI anyone on the team can use. Batch uploads, because big importers drop 200–300 applications at once.
- **Dave (senior agent, 28 years):** judgment, not literal matching. `STONE'S THROW` on the label and `Stone's Throw` in the application are the same brand. Don't make his queue harder.
- **Jenny (junior agent):** the government warning is the opposite case — it must be word-for-word exact, with "GOVERNMENT WARNING" in all caps and bold. Bonus: handle imperfect photos.
- **Marcus (IT):** a standalone proof of concept. No COLA integration, nothing sensitive stored, and the production network blocks many outbound domains.

HOW I IMPLEMENTED IT

- **Speed (Sarah).** One vision call reads a product's front and back together; the warning's second read runs in parallel, so it normally adds no time. A front + back product verifies in roughly 7–10 seconds; single-image products are faster.
- **Simple UI (Sarah).** Two tabs: single label and batch. Drag and drop, a worst-first results table, click-to-zoom label images next to the verdicts. No hunting for buttons.
- **Batch (Sarah).** Drop hundreds of photos. Files pair into products by filename (`oldtom_front.jpg` + `oldtom_back.jpg`), an Excel workbook supplies each product's expected values, and every product runs in parallel as its own request — one bad product becomes an error row, never a dead batch.
- **Judgment (Dave).** Three verdicts, not two. Brand and class fuzzy-match, so case, punctuation, and word order don't fail; a 1–2 character difference goes to *needs review* instead of auto-passing (could be a real typo); net contents is compared by parsed volume — `750 mL` vs `0.75 L` is a review, not a fail.
- **Exact warning (Jenny).** Fail-closed. A dedicated second model transcribes the warning, and wording and capitalization are judged deterministically from that transcription — reworded or title-case warnings fail. Bold is an observation, so at worst it routes to review for human eyes; it never silently passes.
- **Imperfect photos (Jenny).** A low-confidence read downgrades a pass to *needs review* with an image-quality note. The tool asks for human confirmation instead of rejecting.
- **Standalone PoC (Marcus).** Expected values are typed in or loaded from the Excel template; nothing is stored server-side. The one external dependency (the OpenAI API) is documented as what a TTB-network deployment would have to allowlist or replace.

ASSUMPTIONS MADE

- The reviewer supplies the expected values; production would pull them from COLA.

- Scope is what a photo can prove. Standard-of-fill sizes, type-size minimums, and "same field of vision" layout rules are out; disclosures whose triggers aren't visible (sulfites, FD&C Yellow #5, …) are transcribed for the reviewer, not judged.
- A product is at most 4 images; the client downscales big photos before upload (serverless request-body cap).
- The vision model isn't perfectly deterministic, so nothing non-exact ever auto-passes — borderline reads land in *needs review*.
- It's a screening aid for a human reviewer, not a final legal determination.

WHY THIS ARCHITECTURE

- **Why "the model reads, Python judges."** If the model judged compliance, the rules would live in a prompt: untestable, unauditable, and they drift when the model changes. Here the model only transcribes; every pass/fail rule is plain Python (`verification.py`) with the regulatory constants in one file (`config.py`). The rules are unit-tested offline and survive a model swap. The extractor also never sees the applicant's values, so the model can't agree with itself into false passes.
- **Why OpenAI and `gpt-5.4-mini`.** The job needs a vision model that returns dense label text in a fixed JSON schema. Candidates were compared on a repeated-run (5×) stability benchmark; `gpt-5.4-mini` gave the most consistent reads at low latency and cost. It's an env var (`EXTRACTION_MODEL`), so swapping models needs no code change.
- **Why a second model (`gpt-4.1`) just for the warning.** Highest-stakes field, and the main read wasn't good enough: a focused warning-only read scored 100% on ground-truth labels (60/60) vs 70% for the main read. A different model family is deliberate — a same-family second reader repeated the main model's misreads. It runs in parallel, so the accuracy normally costs no wall-clock time, and `WARNING_SUPPLEMENT_MODEL=""` turns it off.
- **Why RapidFuzz.** Dave's matching problem is deterministic string scoring, not an AI judgment call. A fuzzy scorer is fast, repeatable, threshold-tunable, and its verdicts are pinned in unit tests.
- **Why FastAPI.** The engine is Python — best OpenAI SDK support, and regulatory logic belongs in pytest-able plain code — so the API layer should be Python too; porting it would mean two diverging copies of the rules. FastAPI adds exactly what a thin layer needs (Pydantic validation, multipart uploads, auto-generated docs) and runs as a Vercel serverless function with no server to manage.
- **Why Next.js + TypeScript.** The UI is interactive (uploads, batch progress, evidence panels), which means React, and Next.js is the React framework with first-class Vercel support. Its rewrites keep frontend and API same-origin in dev and prod — no CORS at all. TypeScript mirrors the API's Pydantic models 1:1, so the contract is typed end to end.
- **Why Tailwind.** Fastest way to a clean, consistent UI without hand-rolling a design system. Pinned to v3 (v4's native build doesn't run on the dev machine; the output is identical for this UI).
- **Why SheetJS, in the browser.** Compliance teams live in Excel, so batch expected values come from a workbook — a ready-to-fill template is downloadable in the app. Parsing client-side keeps the API single-purpose and the workbook off the server.
- **Why Vercel.** The deliverable is a working deployed URL. One Vercel project hosts both the Next.js frontend and the Python function — no separate API host, no CORS, no infrastructure to run. Batch scales on its own because each product is its own serverless invocation.
- **Why offline tests.** `pytest` covers the engine and API glue with the model call mocked; `node:test` covers the batch-grouping and Excel-parsing logic. The suite runs with no network and no API key, so CI (GitHub Actions) runs the tests, the type-check, and the production build on every PR and every push to main — fast and free.